

Project Design Documentation

Version Control

Version: DRAFT v0.2 **Date:** 08/03/2026 **Change Log:** Expanded all sections per template requirements — added key features/MVP scope, full tech stack, architecture diagram, 12-week SDLC-aligned milestone table, risk register, measurable evaluation criteria, and future considerations. Reframed data-source integrations from live enterprise APIs to schema-matched simulated services for feasibility within a 12-week solo build.

1. Project Title & Version Control

AI-Agentic Triage and Investigation Engine

2. Project Summary

This project is an LLM-orchestrated investigation agent that autonomously triages security alerts: it pulls identity context (Okta/Azure AD-shaped), queries SIEM history (Splunk-shaped), and checks EDR telemetry (CrowdStrike-shaped) before a human analyst ever opens the ticket. Unlike traditional SOAR automation, which follows fixed if-this-then-that playbooks, the agent dynamically decides which investigative steps a given alert actually needs, then correlates the results into a single MITRE ATT&CK-mapped timeline and a natural-language risk summary for human review.

3. Problem Statement / Use Case

Tier 1 SOC analysts spend most of their time on repetitive investigative legwork — checking a user's login history, verifying whether a source IP or device is unusual, seeing if an endpoint has related process activity — before they can even judge whether an alert is a genuine threat. Traditional SOAR platforms automate *fixed* sequences of steps, but they break down when an alert doesn't fit the template, and they don't explain *why* something looks risky in language a human can audit quickly.

This tool is intended for Tier 1/Tier 2 SOC analysts and SOC leads, particularly at organizations too small to justify a commercial agentic-SOC contract (comparable products in this space in 2026 include Dropzone AI, Prophet Security, and Radiant Security), or as an internal proof-of-concept for a security engineering team evaluating whether to build or buy this capability. The tool's objective is to codify a premeditated, judgment-informed course of action for common attack patterns — credential stuffing, brute force, phishing-driven account compromise, ransomware precursors — so the agent narrows a flood of raw telemetry down to the specific, correlated evidence a human actually needs to make a fast, defensible decision. This is where the project draws directly on an emergency-response background: the hard part isn't the API plumbing, it's encoding triage judgment under incomplete information and time pressure.

4. Goals and Objectives

By the end of the 12-week development period, the project must accomplish:

1. **Select and integrate an LLM capable of agentic tool-calling** (Claude or GPT-family, via LangGraph orchestration) that can plan and execute a multi-step investigation rather than follow a fixed script, and deploy that agent against realistic simulated alert data.
2. **Stand up a safe, self-contained test environment** — a VPS-hosted deployment with mocked identity/SIEM/EDR services (schema-matched to real Okta, Splunk, and CrowdStrike APIs) so the system can be exercised end-to-end without requiring costly enterprise tool access, and so it's reproducible for a demo or code review.
3. **Produce a defensible, measured outcome**, not just a working demo: build a labeled test set of alerts with known ground truth, run the agent against it, and be able to state plainly how I "accomplished [X — autonomous multi-source triage] through [Y — an LLM agent orchestrated with LangGraph and human-approval guardrails], resulting in [Z — a measured reduction in investigation time and a measured triage accuracy rate against my own test set]."

5. Key Features / Functions

MVP (must-have for a complete, demoable v1):

- Alert ingestion: accepts a structured JSON alert (modeled on Splunk ES notable-event / Sigma-rule output format).
- Dynamic investigation planning: the agent decides which of identity, SIEM history, and EDR context it needs to pull, based on alert type — not a hardcoded sequence.
- Three tool-calling connectors: identity context lookup, SIEM historical query, EDR telemetry pull (all against mocked, schema-matched APIs — see Section 6).
- Correlation and timeline construction: merges findings from all three sources into one chronological incident narrative.
- MITRE ATT&CK mapping: tags the incident with relevant tactics/techniques (e.g., Initial Access, Credential Access, Persistence).
- Natural-language report synthesis: translates the technical timeline into a business-risk summary a non-specialist could act on.
- Auditable reasoning chain: every agent decision (why it queried a given source, why it flagged a given risk) is logged and traceable, not just the final verdict.
- Human-in-the-loop approval gate: the agent can *recommend* a containment action (e.g., isolate host, disable account) but cannot execute it — a human must explicitly approve.

- Pre-established scenario library: a set of known attack patterns (credential stuffing, brute force, phishing-driven compromise, ransomware precursor behavior) with expected investigation paths, used both to guide agent behavior and as the ground truth for evaluation.

Explicitly out of scope for the 12-week MVP (see Section 11 — Future Considerations): live enterprise API integrations, multi-tenant support, autonomous threat hunting, a polished custom frontend beyond a simple dashboard, automatic (non-approved) execution of any containment action.

6. Tech Stack and Tools

Category	Tool	Purpose
Language	Python	Core implementation language for agent logic, mock APIs, and data processing
Agent orchestration	LangGraph (built on LangChain)	Stateful, cyclic multi-agent orchestration with native human-in-the-loop pause/resume — the right fit over a linear LangChain chain or the deprecated AgentExecutor, since the workflow needs branching, looping, and an approval checkpoint
LLM	Anthropic Claude API (or OpenAI API as a fallback/comparison)	Powers the supervisor agent's planning, the correlation agent's reasoning, and the report-synthesis agent's natural-language output; selected for tool-calling support and structured-reasoning reliability

Mock identity service	FastAPI service, schema-matched to Okta/Azure AD public API structure	Serves synthetic user records: login history, geolocation, MFA status, privilege level
Mock SIEM	Splunk Free (local, single-user, no license cost) or a FastAPI mock, populated with the open Security-Datasets project (OTRF/Security-Datasets) or the Mordor dataset	Provides realistic, MITRE-labeled historical log data for the agent to query
Mock EDR	FastAPI service shaped like the CrowdStrike Falcon API, populated with synthetic or open EDR telemetry (e.g., DARPA OpTC-style process-tree data)	Serves endpoint process/network telemetry for correlation
Ground-truth labeling reference	Atomic Red Team (technique-to-log mappings)	Used to label the test alert set with known MITRE ATT&CK techniques so evaluation results are defensible
Database	SQLite (MVP) or Postgres (if time allows)	Stores investigation state, reasoning-chain logs, and the test-set results
Deployment	VPS (e.g., Hetzner or DigitalOcean) + Docker	Hosts the full stack for safe local/remote testing and demo access
Dashboard	Streamlit (lightweight)	Displays alert queue, investigation reports, reasoning chains, and the approval gate UI

Observability	LangSmith (LangChain's tracing layer)	Debugging and step-by-step replay of agent decisions during development
---------------	--	---

7. Architecture / Workflow Diagram



This is deliberately drawn as a **SOC investigation workflow first, an AI system second** — the framing an analyst or SOC lead would recognize from a runbook, not a diagram of AI infrastructure. The stages map directly onto how a human analyst already works a case:

- **Alert received** — a SIEM detection fires, same as today.
- **Triage decision engine** — scopes what the case actually needs investigated. This is where AI does the work a Tier 1 analyst currently does by hand — deciding *which* checks matter for *this* alert, instead of running a fixed checklist regardless of relevance.
- **Identity & access / Historical activity / Endpoint evidence** — the three evidence pillars any real investigation pulls together (who is this user, what's their history, what did the endpoint actually do), gathered in parallel rather than one at a time.
- **Incident timeline** — the evidence is stitched into a single, chronological narrative and tagged against MITRE ATT&CK, the same taxonomy analysts already use to describe attacker behavior.
- **Analyst report** — a risk-framed writeup with a full evidence trail, so the analyst is reviewing a case file, not trusting a black-box score.
- **Analyst decision** — the only step with real authority. The system can recommend closing, escalating, or containing; only the analyst can authorize it.

Input: a raw, structured security alert (JSON). **Output:** an investigation-ready case report — verdict, ATT&CK-mapped timeline, evidence trail, and (if applicable) a proposed, not executed, containment action awaiting analyst sign-off.

8. Timeline / Weekly Milestones

Week(s)	SDLC Phase	Tasks
---------	------------	-------

1–2	Planning & environment setup	Finalize alert schema; stand up mock Okta/Splunk/CrowdStrike APIs; provision VPS; pull and prepare Security-Datasets / Atomic Red Team reference data
3–4	Core development — foundation	Build LangGraph supervisor agent with basic tool-calling to the three mock APIs; get raw data flowing end-to-end (no reasoning polish yet)
5–6	Core development — correlation	Build correlation/timeline logic and MITRE ATT&CK mapping layer
7–8	Core development — synthesis	Build report-synthesis agent (natural-language, business-risk framing) and structured reasoning-chain logging
9	Core development — guardrails	Implement the human-in-the-loop approval gate as an actual pause/resume in the LangGraph state machine
10	Testing & evaluation	Build a labeled test set of 20–50 synthetic alerts with known ground truth; run full evaluation; record accuracy and time-to-triage metrics

11	Hardening & polish	Cost-optimization pass (pre-filtering before LLM calls, tiered model use, caching); build the Streamlit dashboard; harden guardrails based on Week 10 findings
12	Release & documentation	Finalize documentation, demo video/ walkthrough, portfolio writeup with real (not estimated) XYZ metrics

9. Risks and Risk Mitigation

Risk	Category	Mitigation
No affordable access to real Okta/Splunk/CrowdStrike enterprise APIs	Access to tools	Build schema-matched mock services instead of live integrations; the connector code stays swappable for real APIs later, and this is explicitly called out as intentional scoping, not a shortcut
LLM agent produces inconsistent or hallucinated reasoning across runs	Technical difficulty	Use structured/JSON-schema-constrained outputs for intermediate agent steps; reserve free-text generation for the final human-facing summary only; validate against the labeled test set
Scope creep — trying to match commercial products (Dropzone, Prophet, Radiant) feature-for-feature	Scope	Explicit MVP/out-of-scope split (Section 5); de-prioritized features are documented in Section 11 rather than silently dropped

Publishing an unverified "% of alerts automated" claim that can't be defended in an interview	Missing requirements / credibility	Numbers in the final writeup are pulled only from the self-built, ground-truth-labeled test set (Week 10), not estimated or borrowed from vendor marketing
Token/API cost overruns from sending raw logs to the LLM	Technical / cost	Pre-filter and aggregate logs in Python before any LLM call; tiered model use (cheap model for the initial triage gate, larger model only for alerts that pass it); cache repeated lookups within a session
Agent takes a real containment action without authorization	Security	Human-in-the-loop approval gate is implemented as a hard pause in the LangGraph state machine, not a UI suggestion — the agent has no code path to execute containment actions directly
Time overrun on any single phase given the 12-week window	Time	Weekly milestones are sequenced so a late Week 5–6 doesn't block Week 9's guardrail work — correlation and synthesis logic can proceed on partial data if needed, with polish (Week 11) as the flexible buffer

Open datasets (Security-Datasets, Mordor, DARPA OpTC) don't cover every scenario in the pre-established scenario library	Data	Scenario library is built after reviewing available dataset coverage (Week 1–2), not designed first and forced onto data that doesn't exist
--	------	---

10. Evaluation Criteria

The project will be considered complete and successful when:

1. **Functional completeness:** the agent can take a raw JSON alert as input and produce, without manual intervention, a complete MITRE ATT&CK-mapped timeline, a natural-language risk summary, and an auditable reasoning chain — for at least the four pre-established scenario types (credential stuffing, brute force, phishing-driven compromise, ransomware precursor).
2. **Measured accuracy:** the agent's triage verdicts are evaluated against a self-built, ground-truth-labeled test set of 20–50 synthetic alerts, with the resulting accuracy rate (and false-positive/false-negative breakdown) explicitly reported — not estimated.
3. **Measured efficiency:** average agent investigation time (and, where feasible, token cost) is measured directly and compared against an estimated manual-triage baseline, producing a real, citable time-savings figure for the portfolio writeup.
4. **Guardrail integrity:** in 100% of test cases involving a proposed containment action, the system correctly halts and requires explicit human approval before any such action could be executed — verified by test cases specifically designed to try to trigger unauthorized execution.
5. **Demonstrability:** the full system runs end-to-end on the VPS deployment and can be demoed live (alert in → report out) without manual setup steps beyond starting the service, and is documented well enough that a reviewer (interviewer, instructor) can understand the architecture without a live walkthrough.

11. Future Considerations

(Explicitly separated from the 12-week MVP scope above.)

- Live integrations with real Okta, Splunk, and CrowdStrike tenants, once access is available (the current mock connectors are schema-compatible, so this is a swap, not a rebuild).
- Autonomous threat-hunting mode (proactively searching for undetected compromise, rather than only responding to received alerts) — this is a v2 feature even for the commercial products in this space.

- Multi-tenant support for use across multiple organizations/environments.
- Expanded scenario library beyond the initial four patterns, informed by real-world alert data once available.
- A more polished, production-grade frontend beyond the Streamlit dashboard.
- Fine-tuning or prompt-optimization work specifically on the report-synthesis agent, once enough real usage data exists to know where its explanations fall short.
- Formal cost-per-investigation benchmarking against commercial agentic-SOC platform pricing, to support a build-vs-buy business case.